API Contracts — MVP-aligned

API Contracts — MVP-aligned

Revision: 1 **Last modified:** 2026-07-05T15:00:00Z **Status:** Draft — consolidated MVP-aligned API contracts; subordinate to `docs/research/mvp/final/SPECIFICATION.md`.

Scope: Aligned API contracts for HelixVPN MVP: agent⇄control-plane protobuf, session management, tunnel/UI events, telemetry, and REST/WS/SSE surface boundaries.

---

1. Proto-first rule

All language clients are **generated** from `submodules/helix_proto/`. No hand-written parallel structs are allowed. The canonical source files are:

Package	File	Contract
`helix.coordinator.v1`	`proto/helix/coordinator/v1/coordinator.proto`	Agent⇄control-plane
`helix.session.v1`	`proto/helix/session/v1/session.proto`	Auth/session
`helix.tunnel.v1`	`proto/helix/tunnel/v1/tunnel.proto`	Data-plane/ tunnel events
`helix.ui.v1`	`proto/helix/ui/v1/ui.proto`	UI state
`helix.telemetry.v1`	`proto/helix/telemetry/v1/telemetry.proto`	Telemetry/ observability

Code generation is driven by `submodules/helix_proto/buf.yaml` and `buf.gen.yaml`.

---

2. Agent⇄control-plane contract (`helix.coordinator.v1`)

This is the spine contract, served over **Connect** (gRPC / gRPC-Web / Connect protocol) on the same HTTPS listener as the Gin REST API.

2.1 Service

```
service Coordinator {
  rpc Enroll(EnrollRequest) returns (EnrollResponse);
  rpc WatchNetworkMap(WatchRequest) returns (stream MapUpdate);
  rpc AdvertisePrefixes(AdvertiseRequest) returns
        (AdvertiseResponse);
  rpc ReportStatus(StatusReport) returns (StatusAck);
}
```

2.2 Enrollment

- `Enroll` is the **only unauthenticated RPC**.
- Authenticates by a single-use, short-lived enroll token (hashed server-side).
- Device sends its **WireGuard public key only**; private key never leaves device.
- Returns `device_id`, `overlay_ip`, short-lived mTLS cert, and `GatewayInfo`.

2.3 WatchNetworkMap

- Agent opens one server-streaming RPC over mTLS.
- First frame is a `snapshot` OR catch-up `deltas` based on `known_version`.
- Subsequent frames are live `deltas` and `keepalives`.
- Peers are **already policy-filtered** (need-to-know).
- Convergence SLO: p99 event→delta-on-wire < 1 s.

2.4 AdvertisePrefixes

- Connector-only RPC.
- `cidrs` is the **complete** set the connector serves (declarative, idempotent).
- Overlaps return as advisory `conflicts`, not hard rejects (resolved by 4via6).

2.5 ReportStatus

- Carries **only** `device_id`, `transport`, `rtt_ms`.

- No bytes, flows, destinations, or packet counts — no-logging by construction.

2.6 Error taxonomy

Condition	Connect code	Retriable?
Bad/used/expired enroll token	`Unauthenticated`	No
Missing/invalid mTLS cert	`Unauthenticated`	No
Bad field (wg pubkey length, empty token, unspecified kind)	`InvalidArgument`	No
Device ID / cert mismatch	`PermissionDenied`	No
Device revoked	`PermissionDenied`	No
Non-connector advertises prefixes	`PermissionDenied`	No
Slow consumer / stream cap	`ResourceExhausted`	Yes
Coordinator not ready / backend down	`Unavailable`	Yes
Context cancel / deadline	`Canceled` / `DeadlineExceeded`	Yes

---

3. Session contract (`helix.session.v1`)

Used by Console (web/desktop) which has no mTLS device identity. Agents use `Coordinator.Enroll` instead.

```
service Session {
  rpc Authenticate(AuthenticateRequest) returns
        (AuthenticateResponse);
  rpc ValidateToken(ValidateTokenRequest) returns
        (ValidateTokenResponse);
  rpc RevokeToken(RevokeTokenRequest) returns
        (RevokeTokenResponse);
}
```

- `Authenticate` exchanges an OIDC ID token for a short-lived session token and a longer-lived refresh token.
- `ValidateToken` is called by the API gateway on REST/Connect requests.
- `RevokeToken` supports sign-out and device revoke.

---

4. Tunnel contract (`helix.tunnel.v1`)

Mirrors the Rust `TunnelEvent` vocabulary so Dart/Rust generated bindings stay aligned.

4.1 State machine

```
enum TunnelState {
  IDLE, CONNECTING, HANDSHAKING, CONNECTED,
  RECONNECTING, DISCONNECTING, DISCONNECTED,
  KILL_SWITCH_ACTIVE, ERROR
}
```

4.2 Events

```
message TunnelEvent {
  oneof event {
    StateChanged  state_changed = 1;
    ConnectedInfo connected     = 2;
    Disconnected  disconnected  = 3;
    ErrorInfo     error         = 4;
    StatsUpdate   stats         = 5;
  }
}
```

4.3 Commands

```
message TunnelCommand {
  oneof command {
    StartCommand start = 1;
    StopCommand  stop  = 2;
    PinTransport pin   = 3;
    SelectExit   exit  = 4;
  }
}
```

Dart calls into Rust FFI with these commands; Rust emits `TunnelEvents` back.

---

5. UI contract (`helix.ui.v1`)

Normalized UI state shared across Access, Connector, and Console.

```
message UiState {
  ConnectionCard connection   = 1;
  NetworkList    networks     = 2;
```

```
  SettingsCard   settings     = 3;
  Notification   notification = 4;
}
```

The Flutter layer maps `helix.tunnel.v1.TunnelEvent` + control-plane API responses into `UiState`.

---

6. Telemetry contract (`helix.telemetry.v1`)

Aggregate, privacy-preserving metrics and operational logs.

```
service Telemetry {
  rpc SubmitMetrics(MetricsBatch) returns (MetricsAck);
  rpc SubmitLog(LogEntry) returns (LogAck);
}
```

Rules:

- Metrics are counters/gauges/histograms only.
- Labels use a **whitelisted key set**; no IPs, device fingerprints, or destinations.
- Log `fields` MUST NOT contain packet contents, per-connection state, or SNI.

Example allowed metric:
`helix_transport_escalations_total{transport="masque-h3",region="eu"}`.

---

7. REST / WebSocket / SSE boundaries

The control plane also exposes a Gin REST API and live WebSocket/SSE endpoints for Console. Those endpoints consume the same protobuf messages internally but serialize as JSON where appropriate.

Surface	Used by	Notes
Connect / gRPC	Native agents	`helix.coordinator.v1`
gRPC-Web	Browser Console	Same service
REST (OpenAPI)	Console CRUD	To be specified; uses proto JSON mapping
WebSocket/ SSE	Console live state	Streams `UiState` and control-plane events

Important: the REST/WS/SSE OpenAPI schema is a follow-up deliverable. The protobuf messages defined here are the authoritative data shapes it must reuse.

---

8. Versioning

- `v1` field numbers are frozen.
- Additive changes use new field numbers; no renumbering.
- A `v2` package is warranted only for semantic breaking changes (redefined field meaning, removed RPC, replaced streaming model).
- `buf breaking --against '.git#branch=main'` gates every proto change.

---

9. Generation commands

Go (available in this environment)

```
cd submodules/helix_proto
buf lint
buf generate
```

This emits `gen/go/helix/...` message stubs and `...connect/` service stubs.

Dart (requires `protoc-gen-dart`)

```
# Install protoc-gen-dart, then uncomment the Dart plugin in
        buf.gen.yaml
buf generate
```

Rust (requires `protoc-gen-prost` + `protoc-gen-tonic`)

```
# Install the plugins, then uncomment the Rust plugins in
        buf.gen.yaml
buf generate
```

---

10. Links

- Protobuf source: `submodules/helix_proto/proto/helix/...`
- Buf config: `submodules/helix_proto/buf.yaml`, `submodules/helix_proto/buf.gen.yaml`
- Control-plane spec: `docs/research/mvp/final/02-control-plane.md`

- Protobuf spec: `docs/research/mvp/final/v03-control-plane/protobuf-spec.md`
- Data-plane spec: `docs/research/mvp/final/01-data-plane.md`
- System architecture: `docs/research/mvp/final/implementation/02-system-architecture/README.md`
- Decoupling plan: `docs/research/mvp/final/implementation/02-system-architecture/decoupling-plan.md`
- Alignment report: `docs/reviews/mvp-final/findings/phase3-code-spec-alignment.md`